```python
blocks = list(map(int, input("Enter memory blocks: ").split()))
processes = list(map(int, input("Enter process sizes: ").split()))

allocation = [-1] * len(processes)

for i in range(len(processes)):
    worst = -1

    for j in range(len(blocks)):
        if blocks[j] >= processes[i]:
            if worst == -1 or blocks[j] > blocks[worst]:
                worst = j

    if worst != -1:
        allocation[i] = worst
        blocks[worst] -= processes[i]

print("\nProcess\tSize\tBlock")

for i in range(len(processes)):
    if allocation[i] != -1:
        print("P" + str(i + 1), "\t", processes[i],
              "\t", allocation[i] + 1)
    else:
        print("P" + str(i + 1), "\t", processes[i], "\tNot Allocated")
```

Output

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Py
ssssssssssssssssssssssssssssssssssssssssssssssss
Enter memory blocks: 2
Enter process sizes: 2

Process Size    Block
P1      2        1
```

```python
blocks = list(map(int, input("Enter memory blocks: ").split()))
processes = list(map(int, input("Enter process sizes: ").split()))

allocation = [-1] * len(processes)

for i in range(len(processes)):
    best = -1

    for j in range(len(blocks)):
        if blocks[j] >= processes[i]:
            if best == -1 or blocks[j] < blocks[best]:
                best = j

    if best != -1:
        allocation[i] = best
        blocks[best] -= processes[i]

print("\nProcess\tSize\tBlock")

for i in range(len(processes)):
    if allocation[i] != -1:
        print("P" + str(i + 1), "\t", processes[i],
              "\t", allocation[i] + 1)
    else:
        print("P" + str(i + 1), "\t", processes[i], "\tNot Allocated")
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Pytho
ssssssssssssssssssssssssssssssssssssssssssssssssssssss
Enter memory blocks: 3
Enter process sizes: 32

Process Size    Block
P1      32      Not Allocated
>>>
```

```python
blocks = list(map(int, input("Enter memory blocks: ").split()))
processes = list(map(int, input("Enter process sizes: ").split()))

allocation = [-1] * len(processes)

for i in range(len(processes)):

    for j in range(len(blocks)):

        if blocks[j] >= processes[i]:

            allocation[i] = j
            blocks[j] -= processes[i]

            break

print("\nProcess\tSize\tBlock")

for i in range(len(processes)):

    if allocation[i] != -1:
        print("P" + str(i + 1), "\t", processes[i],
              "\t", allocation[i] + 1)
    else:
        print("P" + str(i + 1), "\t", processes[i],
              "\tNot Allocated")
```

OUTPUT

```
>>>
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Python/P
sssssssssssssssssssssssssssssssssssssssssssssssssssssss.
Enter memory blocks: 3
Enter process sizes: 2

Process Size    Block
P1      2       1
>>>
```

```python
import os

filename = "sample.txt"

# Create and write file
with open(filename, "w") as file:
    file.write("Operating Systems Lab\n")
    file.write("Python File Management")

print("File created successfully")

# Get file information
size = os.path.getsize(filename)

print("File Name:", filename)
print("File Size:", size, "bytes")

# Read file
with open(filename, "r") as file:
    data = file.read()

print("\nFile Content:")
print(data)

# Rename file
newname = "new_sample.txt"

os.rename(filename, newname)

print("File renamed successfully")

# Delete file
os.remove(newname)

print("File deleted successfully")
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Pr
sssssssssssssssssssssssssssssssssssssssss
File created successfully
File Name: sample.txt
File Size: 45 bytes

File Content:
Operating Systems Lab
Python File Management
File renamed successfully
File deleted successfully
```

asssssssssssssssssssssssssssssssssssssssssssssssssssssssssssssss.py - C:/Users/ATHIN/AppData/Local/Programs/Python

File Edit Format Run Options Window Help

```
import os

filename = "data.txt"

with open(filename, "w") as file:
    file.write("Operating Systems")

print("File created")

# seek()
with open(filename, "r") as file:

    file.seek(11)

    data = file.read()

    print("Data after seek():", data)

# stat()
info = os.stat(filename)

print("File Size:", info.st_size, "bytes")

# Directory listing
print("\nFiles in current directory:")

files = os.listdir(".")

for file in files:
    print(file)
```

OUTPUT

```
data.txt
DLLs
Doc
employees.txt
fah.py
include
Lib
libs
LICENSE.txt
m.py
mm.py
MMM.py
mmmm.py
NEWS.txt
python.exe
python3.dll
python313.dll
pythonw.exe
s.py
Scripts
t.py
tcl
vcruntime140.dll
vcruntime140_1.dll
xs.py
```

```python
import os

filename = "test.txt"

# Create file
with open(filename, "w") as file:
    file.write("Hello Operating Systems")

print("File created")

# Read file
with open(filename, "r") as file:
    print("\nFile Content:")
    print(file.read())

# Append data
with open(filename, "a") as file:
    file.write("\nFile Management")

print("\nData appended")

# Rename file
newname = "new_test.txt"

os.rename(filename, newname)

print("File renamed")

# Delete file
os.remove(newname)

print("File deleted")
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Lo
ssssssssssssssssssssssssssssssssssss
File created

File Content:
Hello Operating Systems

Data appended
File renamed
File deleted
```

```python
import os

print("Files and Directories:")

files = os.listdir(".")

for file in files:
    print(file)
```

OUTPUT

```
data.txt
DLLs
Doc
employees.txt
fah.py
include
Lib
libs
LICENSE.txt
m.py
mm.py
MMM.py
mmmm.py
NEWS.txt
python.exe
python3.dll
python313.dll
pythonw.exe
s.py
Scripts
t.py
tcl
vcruntime140.dll
vcruntime140_1.dll
xs.py
```

```python
filename = input("Enter file name: ")
word = input("Enter word to search: ")

try:

    with open(filename, "r") as file:

        found = False

        for line in file:

            if word.lower() in line.lower():

                print(line.strip())
                found = True

        if not found:
            print("Word not found")

except FileNotFoundError:

    print("File not found")
```

Output

```
= RESTART: C:/Users/ATHIN/AppDa
ssssssssssssssssssssssssssssssss
Enter file name: ass
Enter word to search: if
File not found
```

```python
import threading
import time

buffer = []

empty = threading.Semaphore(5)
full  = threading.Semaphore(0)
mutex = threading.Lock()


def producer():

    for i in range(1, 6):

        empty.acquire()

        mutex.acquire()

        buffer.append(i)

        print("Produced:", i)

        mutex.release()

        full.release()

        time.sleep(1)


def consumer():

    for i in range(1, 6):

        full.acquire()

        mutex.acquire()

        item = buffer.pop(0)

        print("Consumed:", item)
```

Output

```
= RESTART: C:/Users/ATHIN/AppData/Local/
ssssssssssssssssssssssssssssssssssssssss
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Synchronization completed
```

```python
import threading
import time


def task():

    thread = threading.current_thread()

    print("Thread started")
    print("Thread name:", thread.name)

    time.sleep(1)

    print("Thread exiting")


# Create thread
t1 = threading.Thread(target=task)

print("Thread created")

# Start thread
t1.start()

# Check thread identity
if t1 is threading.current_thread():
    print("Same thread")
else:
    print("Different thread")

# Join thread
t1.join()

print("Thread joined")
print("Main program completed")
```

Output

```
= RESTART: C:/Users/ATHIN/AppData/Loc
sssssssssssssssssssssssssssssssssss
Thread created
Thread startedDifferent thread

Thread name: Thread-1 (task)
Thread exiting
Thread joined
Main program completed
|
```